

LLD – Auth Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Authentication module of OpenScholar. It covers API contracts, data model extensions, service logic, security mechanisms, and implementation structure. The module is designed for a Next.js fullstack modular monolith with future migration to NestJS.

2. Scope

The Auth module is responsible for:

- User registration (email + password)
 - Email verification via token link
 - User login using JWT
 - Password reset using token-based flow
 - Role assignment (student/admin)
-

3. API Contracts

3.1 Register User

Endpoint: POST /api/auth/register

Request Body: { "name": "John Doe", "email": "john@example.com", "password": "123456", "role": "student" }

Validation Rules:

- email must be unique
- password length ≥ 6
- role must be one of: student, admin

Response: { "message": "User created. Verification email sent." }

3.2 Login

Endpoint: POST /api/auth/login

Request Body: { "email": "john@example.com", "password": "123456" }

Response: { "accessToken": "<jwt_token>", "user": { "id": "uuid", "name": "John Doe", "role": "student" } }

Error Cases:

- Invalid credentials
 - Email not verified
-

3.3 Verify Email

Endpoint: GET /api/auth/verify?token=<token>

Response: { "message": "Email verified successfully" }

Error Cases:

- Invalid token
 - Expired token
-

3.4 Forgot Password

Endpoint: POST /api/auth/forgot-password

Request Body: { "email": "john@example.com" }

Response: { "message": "Password reset link sent" }

3.5 Reset Password

Endpoint: POST /api/auth/reset-password

Request Body: { "token": "reset_token", "newPassword": "newpassword123" }

Validation Rules:

- password length ≥ 6

Response: { "message": "Password updated successfully" }

4. Data Model Changes

The following fields must be added to the users table:

ALTER TABLE users ADD COLUMN verification_token TEXT; ALTER TABLE users ADD COLUMN verification_token_expiry TIMESTAMP; ALTER TABLE users ADD COLUMN reset_token TEXT; ALTER TABLE users ADD COLUMN reset_token_expiry TIMESTAMP;

5. JWT Design

Token Payload: { "sub": "user_id", "email": "user_email", "role": "student" }

Configuration:

- Algorithm: HS256
 - Expiry: 7 days
 - Secret: stored in environment variable
-

6. Service Logic

6.1 Register

Steps: 1. Validate input 2. Check if email already exists 3. Hash password using bcrypt 4. Generate verification token (UUID) 5. Set token expiry (24 hours) 6. Save user to database 7. Send verification email with token link

6.2 Login

Steps: 1. Retrieve user by email 2. Compare password using bcrypt 3. Check is_verified flag 4. Generate JWT token 5. Return token and user info

6.3 Verify Email

Steps: 1. Find user by verification token 2. Validate token expiry 3. Set is_verified = true 4. Clear verification token fields

6.4 Forgot Password

Steps: 1. Find user by email 2. Generate reset token (UUID) 3. Set expiry (1 hour) 4. Save token 5. Send email with reset link

6.5 Reset Password

Steps: 1. Validate reset token 2. Check expiry 3. Hash new password 4. Update user password 5. Clear reset token fields

7. Middleware Design

7.1 Authentication Middleware

- Extract JWT from Authorization header
- Verify token using secret
- Attach user payload to request context

7.2 Authorization Guard

- Check user role
 - Allow or deny access based on required role
-

8. Email Integration

Verification Link Format: <https://yourdomain.com/api/auth/verify?token=<token>>

Reset Link Format: <https://yourdomain.com/reset-password?token=<token>>

Recommended Tools:

- Nodemailer (SMTP)
 - Resend API (preferred for production)
-

9. Prisma Model (Auth Fields)

```
model User { id String @id @default(uuid()) name String email String @unique password String role String isVerified Boolean @default(false)
```

```
  verificationToken String? verificationTokenExpiry DateTime?
```

```
  resetToken String? resetTokenExpiry DateTime?
```

```
  createdAt DateTime @default(now()) }
```

10. Security Considerations

- Password hashing using bcrypt (salt rounds ≥ 10)
 - JWT secret must be stored securely (environment variable)
 - Token expiry must be enforced
 - Prevent duplicate email registration
 - Validate all inputs
-

11. Edge Cases

- Duplicate email registration
 - Login with unverified email
 - Expired verification token
 - Invalid or expired reset token
 - Multiple password reset requests
-

12. Folder Structure (Next.js)

src/ app/api/auth/register/route.ts app/api/auth/login/route.ts app/api/auth/verify/route.ts app/api/auth/forgot-password/route.ts app/api/auth/reset-password/route.ts

modules/auth/service.ts modules/auth/repository.ts

lib/auth.ts lib/db.ts

13. Future Enhancements

- Refresh token mechanism
 - OAuth integration (Google, GitHub)
 - Multi-factor authentication (MFA)
 - Rate limiting for auth endpoints
 - Session tracking and logout management
-

14. Implementation Notes

- Use Prisma ORM for all database operations
- Use bcrypt for password hashing
- Use jsonwebtoken library for JWT handling
- Ensure API routes are thin and delegate logic to service layer
- Maintain modular structure for future migration to NestJS

15. Acceptance Criteria

- User can register and receive verification email
- User cannot login without verification
- Verified user can login and receive JWT
- User can request password reset and update password
- System rejects invalid or expired tokens
- All endpoints return consistent response formats

End of Document